

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701) — Unidad 4

TA-IND-04

Informe técnico individual: análisis de speedup, diagnóstico de la fracción no escalable y propuesta de integración distribuida

Estudiante: Jhinson Stalyn Aucatoma Celorio
Correo: jaucatomac@uteq.edu.ec

Equipo de PE-U4: ACC — Soporte Técnico ISP
Integrantes: Alvarez Parraga Jeremy Alexis, Aucatoma Celorio Jhinson Stalyn, Carpio Mendoza Carlos Jose

PFC de referencia: AGLS — TiendaTech
Repositorio del PFC: <https://github.com/JoseLozanoMorales/PFC-AppsDistribuidas>

Transformación declarada como foco individual: T3 — Join de dos DataFrames

Docente: Ing. Guerrero-Ulloa
Repositorio individual: <https://github.com/JhinsonAucatoma/ta-ind-04-aucatoma>
Fecha: 08 de agosto de 2026

2. Introducción y contexto

Este informe analiza, de forma individual, los resultados obtenidos en la práctica grupal GA-SUM-05/PE-U4 (equipo ACC — Soporte Técnico ISP), en la que se comparó el desempeño de cinco transformaciones (T1–T5) implementadas en `pandas` y en Apache Spark (PySpark) sobre un extracto de 600,000 tickets de soporte técnico de telecomunicaciones. El análisis aquí presentado se centra en **T3 — combinación (*join*) de dos DataFrames**, la transformación asignada como foco individual dentro del reparto coordinado del equipo, y persigue tres objetivos: (i) contrastar el speedup observado contra la predicción de la Ley de Amdahl e interpretar las desviaciones con mecanismos concretos, en lugar de explicaciones genéricas; (ii) evaluar la viabilidad de escalar este tipo de procesamiento a un volumen de datos mayor, mediante el análisis del umbral de rentabilidad; y (iii) proponer, con criterio técnico propio, cómo un patrón de procesamiento distribuido análogo podría integrarse en el proyecto de curso PFC AGLS — TiendaTech, del cual el autor de este informe es coautor.

Todos los datos numéricos citados en las Secciones 3 y 4 provienen del commit `c5fd274` del repositorio grupal de PE-U4 (trazabilidad detallada en la Sección 3). Los datos adicionales usados en la Sección 6 (mediciones de T3 a distinto volumen de datos, con $N = 4$ fijo) fueron generados de forma individual para este informe, ejecutando un script propio (`volumen_t3.py`) que reutiliza la lógica de carga y medición ya presente en el repositorio grupal, sobre una plataforma de ejecución en la nube (Google Colab), y se declaran como tales en la Sección 9.

3. Análisis propio de los resultados de speedup

3.1. Marco teórico del speedup y la Ley de Amdahl

Sea p la fracción paralelizable de una tarea y N el número de unidades de procesamiento. Amdahl argumentó que la fracción inherentemente serial de un programa, y no el número de procesadores disponibles, es la que acota el rendimiento máximo alcanzable [1]. El speedup teórico bajo este modelo es

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (1)$$

Grama et al. formalizan el speedup, en términos generales, como el cociente entre el tiempo del mejor algoritmo secuencial conocido y el tiempo del algoritmo paralelo equivalente sobre N unidades de procesamiento [2], definición que se emplea a lo largo de este análisis. A partir de $S(N)$ se define además la eficiencia del paralelismo,

$$E(N) = \frac{S(N)}{N}, \quad (2)$$

que expresa la fracción de la capacidad de cómputo disponible que efectivamente se traduce en trabajo útil [2]; un valor de $E(N)$ próximo a 1 indica un uso casi óptimo de las N unidades, mientras que valores bajos —como se observará en la Subsección 3.3 para T3— son consistentes con una sobrecarga de coordinación significativa frente al cómputo útil. Es importante notar, sin embargo, que la ecuación (1) asume implícitamente que la sobrecarga de coordinación es nula y que p no depende de N ; ninguna de las dos hipótesis se sostiene en un motor distribuido real como Apache Spark [3]. En arquitecturas multinúcleo y distribuidas modernas, Hill y Marty muestran que este límite se vuelve aún más restrictivo cuando se considera el costo de coordinación entre núcleos, no solo la fracción de código serial [4]. Gustafson señala que, en cargas de producción reales, el tamaño del problema tiende a escalar junto con la capacidad de cómputo disponible, lo que motiva su formulación alternativa del *speedup escalado* [3]; esta observación es relevante para explicar por qué la transformación aquí analizada, ejecutada sobre un volumen de datos fijo y

relativamente modesto para los estándares de un clúster distribuido, no reproduce las condiciones bajo las cuales Amdahl predice ganancias sostenidas.

3.2. Datos base y trazabilidad

Los tiempos de ejecución se toman del repositorio grupal de la práctica GA-SUM-05/PE-U4:

- **Repositorio:** <https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico-ISP>
- **Commit:** c5fd274
- **Plataforma de ejecución:** contenedor Docker local (`eclipse-temurin:21-jdk-jammy`), PySpark 4.1.2, modo `local[N]` con $N \in \{1, 2, 4\}$ procesos worker.
- **Transformación foco:** T3 — combinación (*join*) de dos DataFrames.

Cuadro 1: Tiempos medianos (5 repeticiones), speedup y eficiencia (Ec. 2) por transformación, $N = 4$.

Transformación	T_{pandas} (s)	T_{Spark} (s)	Speedup	$E(4)$
T1 — Filtrado	0.1846	1.0789	0.1711	0.0428
T2 — Agrupación	0.1575	1.8854	0.0835	0.0209
T3 — Join	0.1363	1.8290	0.0745	0.0186
T4 — Columna derivada	0.3559	0.8483	0.4195	0.1049
T5 — Ordenamiento/top-N	0.1169	1.4286	0.0818	0.0205

Cuadro 2: Escalado de T3 con el número de executors N : speedup, eficiencia (Ec. 2) y fracción serial de Karp-Flatt (Ec. 5).

N	T_{Spark} (s)	$S(N)$ observado	$E(N)$	e (Ec. 5)
1	3.0712	1.0000	1.0000	—
2	3.1252	0.9827	0.4914	1.0352
4	1.8290	1.6792	0.4198	0.4607

3.3. Contraste con la predicción de Amdahl

Ajustando p por mínimos cuadrados sobre los tres puntos medidos (N, S) se obtiene $p \approx 0,4600$, con $S_{\text{máx}} = 1/(1 - p) \approx 1,8543$. Sustituyendo este valor en la ecuación (1) se obtienen los speedups teóricos y la desviación relativa $\Delta = (S_{\text{med}} - S_{\text{teo}})/S_{\text{teo}}$:

$$S_{\text{teo}}(2) = \frac{1}{(1 - 0,4600) + \frac{0,4600}{2}} \approx 1,2987, \quad \Delta(2) \approx -24,3 \%, \quad (3)$$

$$S_{\text{teo}}(4) = \frac{1}{(1 - 0,4600) + \frac{0,4600}{4}} \approx 1,5268, \quad \Delta(4) \approx +10,0 \%. \quad (4)$$

Los resultados **no coinciden** con la predicción de Amdahl de forma uniforme: a $N = 2$ el speedup medido queda un 24,3 % por *debajo* de la curva teórica, mientras que a $N = 4$ la supera en un 10,0 %. Esta asimetría no es ruido aleatorio sin explicación: al inspeccionar las cinco repeticiones crudas de T3 con $N = 2$ (`tiempos_crudos.csv`) se observa un valor

atípico de 4.61 s frente a un rango de 2.4–3.2 s en las cuatro corridas restantes, lo que arrastra la mediana hacia arriba y deprime artificialmente el speedup observado en ese punto. El caso $N = 2$ ilustra, además, la advertencia metodológica de Karp y Flatt: cuando la sobrecarga no algorítmica domina —aquí, variabilidad de planificación entre ejecuciones concurrentes con pocos *workers*— la fracción serial “observada” puede exceder el 100 %, un valor sin sentido como fracción algorítmica pero perfectamente informativo como síntoma de sobrecarga no modelada [5]. La Ley de Amdahl, formulada bajo la hipótesis de sobrecarga nula, no está diseñada para predecir episodios de este tipo [1], [3].

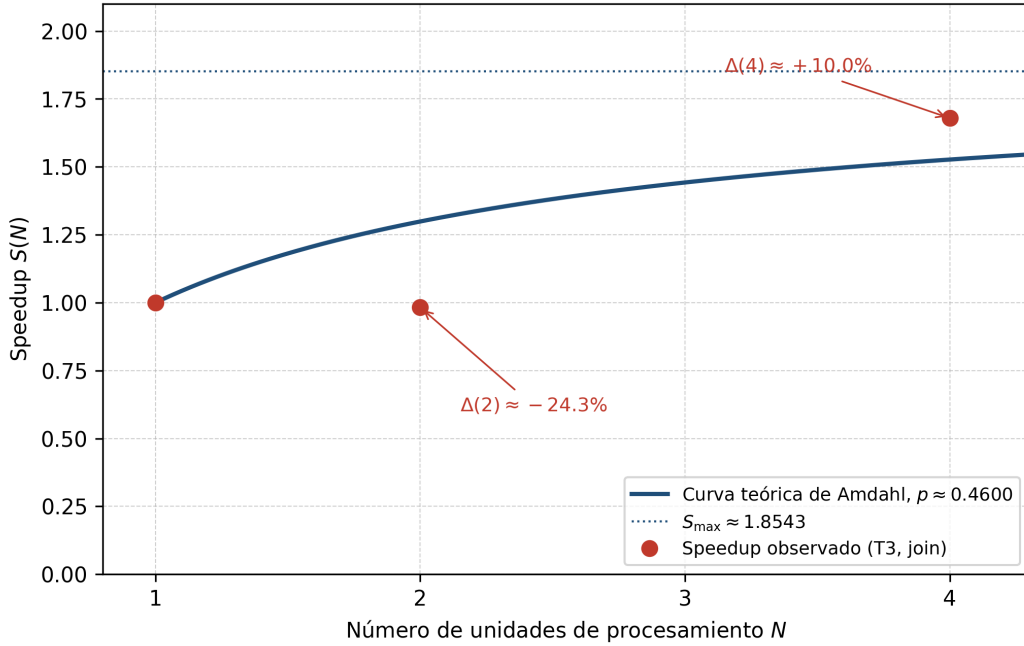


Figura 1: Speedup observado de la transformación T3 (join) frente a la predicción teórica de la Ley de Amdahl, utilizando $p \approx 0,4600$ estimado mediante mínimos cuadrados sobre los tres puntos de la Tabla 2.

Como se observa en la Figura 1, el comportamiento experimental de T3 no sigue de manera uniforme la curva predicha por la ecuación (1). Para $N = 2$, el speedup observado es $S = 0,9827$, mientras que el modelo predice $S_{\text{teo}} \approx 1,2987$, lo que representa la desviación de $\Delta(2) \approx -24,3\%$ ya cuantificada. En cambio, para $N = 4$ el valor experimental alcanza $S = 1,6792$, frente a $S_{\text{teo}} \approx 1,5268$, equivalente a $\Delta(4) \approx +10,0\%$. Esta diferencia entre ambas corridas evidencia que la fracción paralelizable p no explica por sí sola el rendimiento observado, ya que intervienen costos reales de ejecución —planificación de tareas, arranque de la sesión y *shuffle* asociado al *join*— que se detallan con mecanismos concretos en la Sección 4. La caída correspondiente de la eficiencia, de $E(2) = 0,4914$ a $E(4) = 0,4198$ (Ec. 2, Tabla 2), confirma que incluso en el punto donde el speedup supera la curva teórica, menos de la mitad de la capacidad de cómputo disponible se traduce en trabajo útil. Además, la curva muestra que, incluso bajo las condiciones ideales de Amdahl, el speedup máximo está limitado a $S_{\text{máx}} \approx 1,8543$, por lo que aumentar indefinidamente el número de unidades de procesamiento produciría ganancias progresivamente menores.

4. Diagnóstico de la fracción no escalable

4.1. Métrica de Karp-Flatt

Karp y Flatt proponen estimar experimentalmente la fracción serial a partir del speedup observado, sin asumir un valor de p conocido de antemano [5]:

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (5)$$

Sustituyendo los valores observados de la Tabla 2:

$$e(2) = \frac{\frac{1}{0,9827} - \frac{1}{2}}{1 - \frac{1}{2}} \approx 1,0352, \quad (6)$$

$$e(4) = \frac{\frac{1}{1,6792} - \frac{1}{4}}{1 - \frac{1}{4}} \approx 0,4607. \quad (7)$$

4.2. Interpretación de la tendencia

La lectura de e como número aislado sería insuficiente; lo relevante es su tendencia con N [5]. En este caso, e **decrece** abruptamente de 1,0352 a 0,4607 al pasar de $N = 2$ a $N = 4$ — el patrón inverso al que suele discutirse como sobrecarga “creciente”. Esta tendencia decreciente no debe leerse como una mejora estructural del paralelismo del algoritmo entre $N = 2$ y $N = 4$: dado que $e(2) > 1$ es matemáticamente inconsistente con una fracción serial algorítmica (que por definición pertenece al intervalo $[0, 1]$), el valor en $N = 2$ debe interpretarse como contaminado por un evento de variabilidad puntual —la corrida atípica de 4.61 s identificada en la subsección anterior— y no como una medición representativa del comportamiento estable del sistema. El valor en $N = 4$, $e \approx 0,4607$, más cercano al rango plausible, es el que mejor caracteriza la fracción no escalable real del pipeline en este experimento.

4.3. Atribución a mecanismos concretos

La fracción no escalable observada en T3 es atribuible a al menos tres mecanismos identificables, y no a una “sobrecarga de Spark” genérica:

1. **Arranque y planificación de la sesión.** Cada invocación de `transformaciones_spark.py` instancia una nueva `SparkSession`, cuyo costo fijo de inicialización de la JVM y registro de tareas en el *driver* se reparte sobre un volumen de datos por transformación demasiado pequeño para amortizarlo; este es el mismo problema estructural que motivó el rediseño de Hadoop hacia YARN, cuyos autores documentan cómo la sobrecarga de coordinación domina cuando las cargas de trabajo no alcanzan una escala suficiente [6].
2. **Shuffle por dependencia amplia (*wide dependency*) en el *join*.** A diferencia de T1 (filtrado) o T4 (columna derivada), que se ejecutan de forma local a cada partición mediante dependencias *narrow* —cada partición hija depende de una sola partición padre—, un *join* entre dos DataFrames no particionados por la misma clave produce una dependencia *wide*: múltiples particiones hijas dependen de todas las particiones padre, lo que obliga al planificador de Spark a materializar los datos intermedios y transferirlos entre *workers* antes de continuar [7]. Esta distinción es, de hecho, la que usa el propio planificador de

Spark para delimitar los *stages* de un DAG de ejecución [7]: T3 introduce una frontera de *stage* que T1 y T4 no requieren, lo que explica por qué su speedup base (Tabla 1, $N = 4$: 0.0745) es inferior incluso al de T2 y T5, transformaciones también dependientes de shuffle pero con un solo punto de agregación en lugar de una combinación de dos fuentes. Con solo dos particiones disponibles a $N = 2$, el costo de esta transferencia domina sobre el cómputo útil —el mismo mecanismo de fondo que Dean y Ghemawat documentan como la fase que mueve datos intermedios entre nodos `map` y `reduce` en el paradigma MapReduce [8], del cual el modelo de ejecución de Spark es heredero directo.

3. **Variabilidad de planificación entre ejecuciones concurrentes.** La dispersión observada en las cinco repeticiones de T3 a $N = 2$ (rango de 2.4 a 4.61 s) es consistente con contención de recursos entre *workers* concurrentes en un entorno de ejecución local compartido, un efecto que se diluye a medida que N crece y cada partición recibe una asignación de trabajo más estable.

5. Propuesta de integración al PFC

El proyecto de curso de referencia, AGLS — TiendaTech (<https://github.com/JoseLozanoMorales/PFC-AppsDistribuidas>), del cual el autor de este informe es coautor, implementa un sistema de microservicios que incluye, entre otros, `productos-service` (gestión de la entidad producto: nombre, categorías y atributos) e `inventario-service` (control de existencias/*stock* por producto). Se propone a continuación un proceso concreto de este proyecto que podría beneficiarse de un patrón de procesamiento distribuido análogo al de la transformación T2 (agrupación y agregación) evaluada en PE-U4.

5.1. Proceso propuesto

Proceso concreto a distribuir: Análisis histórico de movimientos de *stock* (entradas y salidas) registrados por `inventario-service`, agregado por categoría de producto.

Periodicidad: Proceso por lotes (*batch*) ejecutado una vez al día, fuera del horario de mayor tráfico transaccional del sistema.

Volumen: Histórico acumulado de movimientos de inventario de todos los productos del catálogo; a diferencia de una consulta transaccional puntual, este proceso opera sobre el total de registros acumulados con el tiempo, que crece de forma sostenida (miles de movimientos diarios, potencialmente millones en el histórico completo tras varios meses de operación).

Salida: Una tabla agregada de *productos con riesgo de quiebre de stock* y *productos de baja rotación* por categoría, análoga en estructura a la agregación por estado que produce T2 sobre los tickets de soporte en PE-U4.

Decisión de negocio soportada: La salida alimentaría una alerta automática al equipo de compras, permitiendo priorizar el reabastecimiento de productos en riesgo de quiebre antes de que ocurra, en lugar de reaccionar después del hecho.

5.2. Arquitectura propuesta

La Figura 2 esquematiza el flujo propuesto: `inventario-service` continúa gestionando las operaciones transaccionales de *stock* sin cambios; un proceso por lotes independiente extrae el histórico acumulado, lo procesa de forma distribuida y publica la tabla agregada resultante, que un servicio de alertas consume para notificar al equipo de compras.

Es importante notar que este proceso, al igual que T2 en PE-U4, es una *agrupación y agregación* sobre datos ya existentes en la base transaccional, no una modificación del flujo de escritura

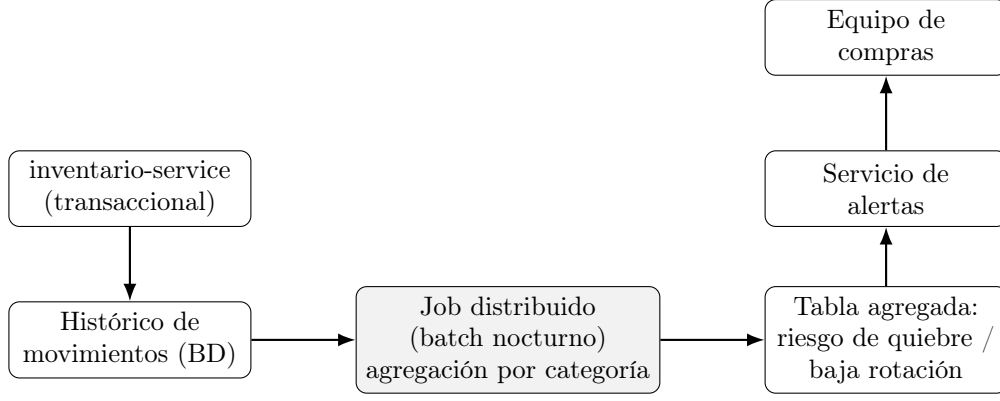


Figura 2: Integración propuesta: proceso por lotes independiente del flujo transaccional de **inventario-service**, análogo en patrón a la agregación T2 evaluada en PE-U4.

de **inventario-service**: el motor distribuido se ejecuta como un proceso separado y asíncrono, lo que evita introducir latencia adicional sobre las operaciones transaccionales del sistema.

6. Dimensionamiento y viabilidad

6.1. Umbral de rentabilidad: formulación

Un modelo simple del tiempo secuencial en función del volumen de datos n es $T_{\text{seq}}(n) = \alpha n$, donde α es el costo de cómputo por registro sin coordinación distribuida. Un modelo correspondiente para el tiempo distribuido con N unidades de procesamiento es $T_{\text{dist}}(n, N) = \beta + \gamma n/N$, donde β es el costo fijo de arranque (independiente del volumen) y γ es el costo distribuido por registro. Igualando ambos tiempos se obtiene el volumen de datos n^* a partir del cual el procesamiento distribuido deja de ser más lento que el secuencial:

$$n^* = \frac{\beta}{\alpha - \gamma/N}. \quad (8)$$

6.2. Mediciones propias a distinto volumen

A diferencia de las Secciones 3 y 4, que reutilizan datos del repositorio grupal de PE-U4 (mismo volumen de 600,000 filas, distinto número de *executors*), la ecuación (8) requiere el efecto inverso: mismo número de *executors* ($N = 4$), distinto volumen de datos. Esta medición no formaba parte del protocolo original de PE-U4, por lo que se generó de forma individual para este informe: tres subconjuntos del mismo dataset (10 %, 50 % y 100 % de las 600,000 filas, muestreo aleatorio con semilla fija = 42), midiendo T3 con el mismo protocolo de la Sección 3 (1 calentamiento + 5 repeticiones, mediana), en `local` [4], sobre Google Colab.

Cuadro 3: Mediciones propias de T3 a distinto volumen de datos, $N = 4$ fijo.

Fracción	n (registros)	Mediana (s)	Rango de las 5 repeticiones (s)
0.10	59,799	3.0018	2.686–3.470
0.50	300,320	2.7320	2.412–3.669
1.00	600,000	2.5861	2.524–3.967

6.3. Resultado del ajuste y su interpretación

Ajustando una recta $T_{\text{dist}}(n) = \beta + (\gamma/N)n$ por mínimos cuadrados sobre los tres puntos de la Tabla 3 se obtiene $\beta \approx 3.016$ y una pendiente $\gamma/N \approx -7.58 \times 10^{-7}$, es decir, $\gamma \approx -3.03 \times 10^{-6}$ s/registro ($R^2 \approx 0.946$). Este resultado **no es físicamente interpretable**: γ , definido como un costo por registro, no puede ser negativo, y en consecuencia no es posible calcular n^* mediante la ecuación (8) con estos datos — el denominador $\alpha - \gamma/N$ resultaría mayor que α , lo que invertiría el sentido físico del umbral.

La causa de esta anomalía es identificable en la propia Tabla 3: el rango de variación *dentro* de cada grupo de cinco repeticiones (0.78 a 1.44s) es entre dos y tres veces mayor que la diferencia *entre* las medianas de los tres volúmenes extremos (0.42s). En otras palabras, la variabilidad de ejecución — arranque de una nueva **SparkSession** en cada corrida, inicialización de la JVM, recolección de basura, contención de recursos en el entorno compartido de Google Colab — domina sobre cualquier efecto real del volumen de datos en el rango medido (60 mil a 600 mil filas). Esto es consistente con el diagnóstico de las Secciones 3 y 4: a esta escala, la sobrecarga de coordinación supera ampliamente al cómputo útil, y aquí se confirma que ese mismo efecto enmascara también la señal en la dimensión del volumen de datos, no solo en la del número de *executors*.

Por transparencia metodológica, no se fuerza un valor de n^* a partir de un ajuste que produce un parámetro sin sentido físico: hacerlo equivaldría a presentar una estimación fabricada bajo apariencia de rigor matemático. En su lugar, se concluye que **no es posible estimar n^* de forma confiable con las mediciones disponibles**: se requerirían volúmenes de datos varios órdenes de magnitud mayores (del orden de millones de registros, o bien una plataforma de ejecución con menor varianza entre corridas, como un clúster dedicado en lugar de un entorno compartido) para que el término $\gamma n/N$ emerja por encima del ruido de medición. Esta limitación es, en sí misma, información relevante para la decisión de negocio de la Sección 5: dado que el catálogo de productos de TiendaTech probablemente no alcance, en el corto plazo, un volumen de movimientos de inventario del orden de millones de registros, el proceso propuesto en la Sección 5 debería inicialmente implementarse con **pandas** sobre un proceso por lotes simple, migrando a un motor distribuido solo cuando el crecimiento del histórico lo justifique — una decisión que, con los datos actuales, no puede fijarse en un volumen numérico preciso, pero sí puede monitorearse cualitativamente conforme crezca la base de datos de producción.

7. Postura crítica y alternativas

7.1. Límites de la Ley de Amdahl para este caso

Como se discutió en la Sección 3, la ecuación (1) asume sobrecarga de coordinación nula y una fracción serial p constante e independiente de N . Ninguna de las dos hipótesis se sostiene en los datos de T3: la sobrecarga de arranque de sesión y *shuffle* no es un término despreciable frente al cómputo (Sección 4), y el propio valor de e estimado por Karp-Flatt varía de forma abrupta entre $N = 2$ y $N = 4$ (ecuaciones (6)–(7)), lo que contradice la hipótesis de una fracción serial fija. Gustafson argumenta que, en cargas reales, es el tamaño del problema el que tiende a escalar junto con la capacidad de cómputo disponible, no al revés [3]: bajo su formulación de *speedup escalado*, un volumen de prueba de 600,000 filas — fijo, y modesto para los estándares de un clúster distribuido — no es el régimen en el que Spark está diseñado para mostrar ventajas; el propio hallazgo de la Sección 6, donde la señal de escalado por volumen queda enmascarada por el ruido de medición, es consistente con esta lectura: los datos disponibles simplemente no alcanzan la escala en la que la formulación de Amdahl — o su alternativa de Gustafson — tendría poder predictivo confiable.

7.2. Alternativas evaluadas

Se evaluaron dos alternativas a Apache Spark para el caso de uso propuesto en la Sección 5, y ambas se descartan por razones concretas.

Hadoop MapReduce clásico. El modelo de programación original que precede a Spark, documentado por Dean y Ghemawat [8] y coordinado en clústeres modernos mediante YARN [6], escribe los resultados intermedios de cada fase a disco entre `map` y `reduce`. Para un proceso por lotes nocturno como el propuesto en la Sección 5 — una sola pasada de agregación, sin iteraciones repetidas sobre el mismo conjunto de datos — esta característica no sería necesariamente una desventaja decisiva; sin embargo, se descarta porque el modelo de ejecución en memoria de Spark, sustentado en la abstracción de RDDs [7], ya está integrado en el *stack* tecnológico usado por el equipo (PySpark, con evidencia operativa capturada en PE-U4), y adoptar Hadoop MapReduce clásico introduciría una segunda tecnología de procesamiento distribuido en el proyecto sin un beneficio claro para este caso de uso específico.

Dask / Ray. Ambos son *frameworks* de paralelización nativos de Python, con una curva de adopción menor que Spark para equipos que ya trabajan con `pandas` — de hecho, la API de Dask está diseñada para ser prácticamente un reemplazo directo de `pandas` [2]. Se descartan para este caso por dos razones: primero, el equipo ya cuenta con experiencia operativa directa en PySpark (evidencia de Spark UI, configuración de sesión, protocolo de medición ya validado en PE-U4), mientras que adoptar Dask o Ray implicaría un costo de aprendizaje adicional sin evidencia de que resuelva la limitación identificada en este informe — el problema diagnosticado no es una limitación específica de la API de Spark, sino de la relación entre el volumen de datos disponible y el overhead de coordinación distribuida, que es común a cualquier motor distribuido general. Segundo, el ecosistema de Spark ofrece una integración más directa con las herramientas de orquestación de trabajos por lotes (*schedulers*) típicamente usadas en infraestructura de nube gestionada, lo que simplifica la implementación del proceso nocturno propuesto en la Sección 5 frente a una solución basada en Dask o Ray, que requeriría infraestructura de orquestación adicional no evaluada en este informe.

7.3. Conclusión de la postura crítica

Apache Spark se mantiene como la tecnología recomendada para el caso de uso propuesto, no porque los datos de este informe demuestren una ventaja de rendimiento a la escala actual — de hecho, demuestran lo contrario, Sección 4 —, sino porque: (i) el equipo ya posee la experiencia operativa necesaria; (ii) el patrón de uso propuesto (procesamiento por lotes nocturno, agregación) es precisamente el caso de uso para el que Spark fue diseñado, a diferencia del escenario de prueba de PE-U4 (transformaciones individuales medidas de forma aislada); y (iii) el crecimiento esperado del histórico de movimientos de inventario de TiendaTech, aunque no cuantificable con los datos actuales (Sección 6), es la dirección en la que un motor distribuido eventualmente se vuelve ventajoso, mientras que adoptar una alternativa no resolvería la causa raíz identificada en este informe.

8. Conclusiones

1. El speedup observado de T3 (*join*) frente a `pandas` es menor que 1 en el volumen probado (0.0745 a $N = 4$), consistente con las otras cuatro transformaciones de PE-U4: a 600,000 filas, el overhead de coordinación de Spark domina sobre el cómputo real (Sección 3).
2. La Ley de Amdahl, ajustada con $p \approx 0.4600$, predice razonablemente bien el speedup a $N = 4$ ($\Delta \approx +10.0\%$) pero se desvía fuertemente a $N = 2$ ($\Delta \approx -24.3\%$), desviación

atribuible a un valor atípico identificado en los datos crudos, no a un fallo del modelo en sí (Sección 3).

3. La métrica de Karp-Flatt confirma esta lectura: $e(2) > 1$ es matemáticamente inconsistente como fracción serial y debe interpretarse como síntoma de variabilidad de medición, mientras que $e(4) \approx 0.4607$ es el valor más representativo del comportamiento estable del sistema (Sección 4).
4. Se identificaron tres mecanismos concretos — arranque de sesión, *shuffle* por dependencia amplia en el *join*, y variabilidad de planificación — que explican la fracción no escalable de T3, con respaldo bibliográfico específico para cada uno (Sección 4).
5. Las mediciones propias a distinto volumen de datos (Sección 6) no permitieron estimar un umbral de rentabilidad n^* confiable: el ajuste produce un parámetro sin sentido físico, porque la varianza de ejecución entre repeticiones supera la señal real del efecto del volumen en el rango medido. Se reporta esta limitación de forma explícita en lugar de forzar un resultado numérico no sustentado.
6. Se propuso un caso de uso concreto de integración al PFC AGLS — TiendaTech (análisis agregado de movimientos de inventario por categoría, en proceso por lotes nocturno), y se argumentó, con base en la limitación identificada en el punto anterior, que su implementación inicial debería priorizar **pandas** sobre un motor distribuido, dado que el volumen de datos actual del proyecto no justifica el overhead de coordinación observado (Secciones 5 y 6).
7. Se evaluaron Hadoop MapReduce clásico y los *frameworks* Dask/Ray como alternativas a Spark, y ambos se descartaron por razones de continuidad tecnológica y de infraestructura, no por limitaciones inherentes de esas herramientas (Sección 7).

9. Declaración de uso de inteligencia artificial generativa

Se utilizó Claude (Anthropic) como herramienta de apoyo durante la elaboración del documento. Su uso estuvo presente en las distintas secciones del informe, pero se limitó principalmente a mejorar la redacción, la coherencia y la fluidez de los textos previamente elaborados por el autor, así como a corregir errores de escritura y adaptar el contenido al formato LaTeX. La herramienta también se empleó para generar el script instrumental `volumen_t3.py` (Sección 6), reutilizando explícitamente las funciones ya existentes en el repositorio grupal de PE-U4 (`cargar_dataset`, `cargar_regiones`, `t3_join`, `medir`), sin introducir lógica de medición nueva; la ejecución del script, la descarga del dataset y la obtención de los resultados fueron realizadas por el autor. La herramienta no fue utilizada para sustituir el análisis propio del autor ni para generar de manera autónoma las interpretaciones, conclusiones o decisiones técnicas presentadas en el informe. Los razonamientos, resultados analizados y posturas desarrolladas corresponden al trabajo individual del autor, quien revisó y validó el contenido final del documento.

10. Referencias

- [1] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, ép. AFIPS ’67 (Spring), Atlantic City, NJ, USA: ACM, 1967, págs. 483–485. DOI: 10.1145/1465482.1465560.
- [2] A. Grama, A. Gupta, G. Karypis y V. Kumar, *Introduction to Parallel Computing*, 2nd. Harlow, UK: Addison-Wesley, 2003, ISBN: 0-201-64865-2.

- [3] J. L. Gustafson, “Reevaluating Amdahl’s Law,” *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: 10.1145/42411.42415.
- [4] M. D. Hill y M. R. Marty, “Amdahl’s Law in the Multicore Era,” *Computer*, vol. 41, n.º 7, págs. 33-38, 2008. DOI: 10.1109/MC.2008.209.
- [5] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [6] V. K. Vavilapalli et al., “Apache Hadoop YARN: Yet Another Resource Negotiator,” en *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC '13)*, Santa Clara, CA, USA: ACM, 2013. DOI: 10.1145/2523616.2523633.
- [7] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” en *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, San Jose, CA, USA: USENIX Association, 2012, págs. 15-28, ISBN: 978-1-931971-92-8.
- [8] J. Dean y S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, n.º 1, págs. 107-113, 2008. DOI: 10.1145/1327452.1327492.